Introduction to Data Science

A 9-Week Graduate Certificate Course — DBAS 5015

> This textbook accompanies a 9-week course for graduate certificate students in Business Intelligence and Data Analytics. Python and data science concepts are developed together — each week introduces a tool and the problem it solves, so students build technical skill and analytical judgment at the same time.

© 2026 Patrick Dolinger — Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)

You are free to share and adapt this material for any purpose, provided appropriate credit is given to the author and a link to the license is included. See the License section at the end of this document for full terms.

How To Use This Book

Every section follows the same pattern:

1. A data science concept is introduced — what it is, when it applies, and why it matters.
2. The Python implementation is shown — the exact library calls, with comments.
3. A worked example demonstrates the concept on realistic business data, with output.
4. A business interpretation connects the result to a decision a stakeholder would make.

Run every code example yourself. Change the inputs. Break things and fix them. Reading code without running it teaches you almost nothing. Running code — even code you did not write — teaches you what the tool actually does.

The chapters are cumulative. Week 4's regression work assumes you can load and clean a DataFrame (Week 2–3). Week 7's pipeline assumes you understand what each step does individually (Weeks 4–6). Do not skip ahead.

Course Overview

Week	Python / Libraries	Data Science Concept
1	Jupyter, numpy, pandas	The data science workflow; problem framing
2	pandas: cleaning, missing data, dtype handling	Data wrangling; feature identification
3	pandas: encoding, scaling, train/test split	Feature engineering; the ML pipeline
4	scikit-learn: linear regression	Supervised learning — regression; MAE, RMSE, R^2
5	scikit-learn: logistic regression, decision trees	Supervised learning — classification; confusion matrix, precision/recall
6	scikit-learn: random forests, cross-validation	Model tuning; hyperparameter optimization
7	scikit-learn Pipeline, plotly express	End-to-end pipeline; business alignment and model presentation
8	scikit-learn: KMeans, hierarchical clustering	Unsupervised learning; market segmentation
9	Full workflow integration, plotly express	Final Project: EDA → model → insight → communication

Table of Contents

- Chapter 1 — The Data Science Workflow
- Chapter 2 — Data Wrangling
- Chapter 3 — Feature Engineering and the ML Pipeline
- Chapter 4 — Supervised Learning: Regression
- Chapter 5 — Supervised Learning: Classification
- Chapter 6 — Model Tuning and Validation
- Chapter 7 — End-to-End Pipelines and Visualization
- Chapter 8 — Unsupervised Learning: Clustering

- Chapter 9 — Final Project
- License

Chapter 1 — The Data Science Workflow

Data science is applied problem-solving using data. The methods vary depending on the question — regression, classification, clustering, visualization. The workflow is always the same: frame the problem, get the data, prepare it, model it, evaluate it, communicate the result. This chapter establishes that workflow and the tools you will use to execute it.

1.1 What Is Data Science?

Data science is the practice of extracting actionable insight from data. In a business context, that means answering questions like: *Which customers are likely to churn next quarter? What price maximizes revenue without reducing volume? Which transactions are fraudulent?*

Three things separate data science from traditional reporting:

Prediction over description. Traditional reporting tells you what happened. Data science tells you what is likely to happen next. A sales report shows last quarter's revenue by region. A data science model predicts next quarter's revenue by region, with a confidence interval.

Scale. Data science operates on datasets too large to inspect row by row. You cannot read 500,000 customer records. You can build a model that processes all 500,000 in seconds.

Iteration. No model is correct the first time. Data science is a cycle of building, evaluating, and refining — not a single analysis delivered as a final answer.

The Two Main Problem Types

Almost every data science problem in business falls into one of two categories:

Problem Type	Question	Example
Supervised learning	Given inputs, predict an output	Predict customer churn (yes/no) based on usage patterns
Unsupervised learning	Find structure in data with no predefined output	Group customers into segments based on purchasing behaviour

This course covers both. Weeks 4–7 focus on supervised learning. Week 8 covers unsupervised learning — specifically clustering for market segmentation.

1.2 The Data Science Workflow

Every project follows five steps. The order matters. Skipping steps produces unreliable results.

Step	What Happens	Common Mistake
1. Frame the problem	Define the business question and what a useful answer looks like	Starting with data instead of the question
2. Acquire and understand the data	Load the data, inspect its shape, types, and distributions	Assuming the data is what you think it is
3. Clean and prepare	Handle missing values, fix data types, encode categories, scale features	Cleaning after splitting — data leakage
4. Model	Select, fit, and evaluate a model	Choosing a model before understanding the data
5. Evaluate and communicate	Measure performance, interpret results, present findings	Reporting accuracy without business context

This workflow is not linear in practice. You will loop back. Cleaning reveals quality problems that change how you frame the problem. Modeling reveals patterns that send you back to feature engineering. That iteration is normal — it is not a sign that something went wrong.

1.3 Problem Framing

The most common reason a data science project fails is not a bad model — it is a bad question. Before touching data, you need three things:

A decision to support. Data science should inform action. "Understand our customers better" is not a decision. "Identify which customers are at risk of churning so the retention team can intervene" is a decision. The difference matters because it determines what the model needs to predict, what accuracy is good enough, and what the output needs to look like.

A definition of success. What makes the model useful? A fraud detection model that catches 90% of fraud but flags 50% of legitimate transactions is not useful — the false positive rate creates too much friction. Define the threshold before you build.

A label for the target variable. In supervised learning, you need historical data where the outcome is already known. If you want to predict churn, you need records of customers who actually churned. If that data does not exist or is incomplete, supervised learning will not work.

💼 Business Context — Framing as a Data Problem

A retail chain wants to "improve customer retention." That goal becomes a data science problem through a series of decisions:

- *What does retention mean?* — A customer who made a purchase in the last 90 days is active; one who has not is churned.
- *What inputs predict it?* — Purchase frequency, recency, average order value, product category mix, support contacts.
- *What action follows from the prediction?* — Customers flagged as high churn-risk receive a targeted discount offer.
- *What makes the model good enough?* — The retention team can contact 200 customers per week. The model needs to rank customers by churn probability so the team focuses on the highest-risk 200.

Notice that the last point determines the evaluation metric. You do not need 95% accuracy — you need the top 200 predictions to be reliable. Framing drives everything that follows.

1.4 The Python Data Science Stack

You will use six libraries throughout this course. Each has a specific job.

Library	Role	Import Convention
`numpy`	Numerical arrays and math operations — the foundation everything else is built on	`import numpy as np`
`pandas`	Tabular data — loading, cleaning, transforming, aggregating	`import pandas as pd`
`matplotlib`	Low-level static visualization — full control, verbose syntax	`import matplotlib.pyplot as plt`
`seaborn`	Statistical visualization built on matplotlib — cleaner syntax for common plots	`import seaborn as sns`
`plotly express`	Interactive visualization — hover, zoom, export	`import plotly.express as px`
`scikit-learn`	Machine learning — preprocessing, models, evaluation, pipelines	`from sklearn.[module] import [class]`

These libraries are designed to work together. A typical workflow: load with pandas, clean with pandas, visualize with seaborn, model with scikit-learn, present with plotly express.

The scikit-learn API Pattern

scikit-learn uses the same three-method pattern for almost every object — estimators (models), transformers (preprocessors), and pipelines all follow it:

```
model.fit(X_train, y_train)      # learn from training data
model.predict(X_test)            # apply to new data
model.score(X_test, y_test)      # evaluate performance
```

You will see this pattern in Week 4 for the first time. By Week 7, it will be second nature. The consistency is intentional — once you know how one scikit-learn object works, you know how all of them work.

🤖 ML Connection — Why the API Pattern Matters

Scikit-learn's consistent API is what makes pipelines possible. Because every transformer and estimator exposes the same `.fit()` and `.transform()` / `.predict()` methods, you can chain them together into a single object that preprocesses and models in one step. This is not just convenient — it prevents a category of serious errors called data leakage, where information from the test set contaminates the training process. You will see exactly how in Week 7.

1.5 numpy Review

numpy provides the numerical foundation for everything else in the stack. pandas, scikit-learn, and matplotlib all operate on numpy arrays internally. You do not always interact with numpy directly, but understanding arrays makes the rest of the stack more readable.

Arrays

A numpy array is a fixed-type, fixed-size collection of numbers. All elements must be the same type — this constraint is what makes array operations fast.

```
import numpy as np


revenue = np.array([14200, 13850, 16400, 15900, 17200, 18100])
```

Element-Wise Operations

Operations on arrays apply to every element without a loop.

```
# Apply a 5% increase to every value
projected = revenue * 1.05
# [14910.  14542.5  17220.  16695.  18060.  19005.]


# Subtract a fixed cost from every month
net = revenue - 2000
# [12200 11850 14400 13900 15200 16100]
```

Key Statistical Functions

```
np.mean(revenue)      # 15941.67
np.median(revenue)    # 16150.0
np.std(revenue)       # 1545.15
np.min(revenue)       # 13850
np.max(revenue)       # 18100
np.percentile(revenue, 25)   # Q1
np.percentile(revenue, 75)   # Q3
```

Boolean Indexing

Filter an array by a condition — no loop required.

```
# Values above 16000
revenue[revenue > 16000]
# [16400 17200 18100]

# Count of values above 16000
(revenue > 16000).sum()
# 3
```

This pattern — create a boolean mask, apply it to filter — appears constantly in both numpy and pandas.

1.6 pandas Review

pandas is the primary tool for working with tabular data. A DataFrame is a table: rows are observations, columns are variables. A Series is a single column.

Loading Data

```
import pandas as pd

df = pd.read_csv("sales_data.csv")
```

First Steps with a New Dataset

Run these five commands on any new DataFrame before doing anything else.

```
df.shape          # (rows, columns)
df.dtypes         # data type of each column
df.head()         # first 5 rows
df.info()         # column names, types, non-null counts in one view
df.describe()     # summary statistics for all numeric columns
```

`df.info()` is the most useful first step. It shows you the column names, their types, and how many non-null values each has — which immediately reveals missing data and type problems.

Selecting Data

```
# Single column — returns a Series
df["revenue"]

# Multiple columns — returns a DataFrame
df[["region", "revenue", "units_sold"]]

# Rows matching a condition
df[df["region"] == "Northeast"]

# Rows and columns together
df.loc[df["revenue"] > 10000, ["rep", "revenue"]]
```

Adding Calculated Columns

Column arithmetic is element-wise — one line replaces a loop.

```
df["revenue_per_unit"] = df["revenue"] / df["units_sold"]
df["above_target"]     = df["revenue"] > 15000
```

Grouping and Aggregating

```
# Mean revenue by region
df.groupby("region")["revenue"].mean()

# Multiple statistics at once
df.groupby("region")["revenue"].agg(
    count  = "count",
    mean   = "mean",
    median = "median",
    std    = "std"
)
```

💼 Business Context — pandas as the Workhorse

In practice, 60–70% of data science work is pandas work — loading, inspecting, cleaning, and reshaping data before a single model is trained. Students who are fast and confident with pandas move through projects in hours rather than days. Students who are not fluent spend most of their time fighting the data. Weeks 2 and 3 of this course are dedicated to building that fluency.

1.7 Chapter Summary

Concept	Key Point
Data science	Applied problem-solving with data — prediction, scale, and iteration
Supervised learning	Predict an output from inputs using labelled historical data
Unsupervised learning	Find structure in data with no predefined output
The five-step workflow	Frame → acquire → clean → model → evaluate/communicate
Problem framing	Define the decision, the success criteria, and the target variable before touching data
numpy array	Fixed-type numerical collection — element-wise operations, no loops
Boolean indexing	Filter arrays and DataFrames with a condition mask
pandas DataFrame	Tabular data — rows are observations, columns are variables
`df.info()`	First command to run on any new dataset
scikit-learn API	`.fit()` / `.predict()` / `.score()` — consistent across all models

Review Questions

1. A logistics company wants to "use data science to reduce delivery costs." Reframe this as a specific data science problem. What is the target variable? What inputs might predict it? What does a useful model output look like?
2. What is the difference between supervised and unsupervised learning? Give one business example of each that was not used in this chapter.
3. You run `df.info()` on a new dataset and see that a column called `order_date` has dtype `object` instead of `datetime64`. What does this tell you, and what should you do about it before modelling?

4. A numpy array contains the monthly return on investment (ROI) for 12 marketing campaigns. Write the code to: (a) calculate the mean and standard deviation, (b) return only the campaigns with ROI above 0.15, and (c) count how many campaigns had negative ROI.
5. A colleague argues that you should choose your model (e.g., linear regression vs. decision tree) as the first step in a project, then collect data that fits it. What is wrong with this approach?

Chapter 2 — Data Wrangling

Raw data is not clean data. In practice, 60–70% of data science work happens before a model is trained — loading data, diagnosing problems, fixing types, handling missing values, and deciding which columns are worth keeping. This chapter builds the habits and tools for that work.

Every wrangling decision you make shapes what the model sees. A column left as the wrong type causes a runtime error. A missing value filled with zero can introduce a false signal. A duplicate row counted twice inflates a pattern that doesn't exist. Getting this right is not optional — it is the foundation everything else is built on.

2.1 What Is Data Wrangling?

Data wrangling is the process of transforming raw data into a clean, structured form ready for analysis and modelling. It covers four categories of problems:

Problem	Example	Fix
Wrong types	A date column stored as text	Convert with `pd.to_datetime()`
Missing values	Revenue is null for 8% of rows	Drop, impute, or flag
Invalid values	A discount of 150%	Detect and correct or remove
Irrelevant columns	A customer phone number	Drop before modelling

Wrangling is not a one-time step at the start of a project. As you explore data and build models, you will discover new problems that send you back to clean further. That iteration is normal.

💼 Business Context — Why Wrangling Quality Matters

A model trained on dirty data learns the noise, not the signal. If revenue is missing for a specific product line because that line had no sales system in place, filling those nulls with the column median gives the model a fabricated pattern to learn from. If a discount column has a handful of

values above 1.0 due to a data entry error, those values will skew every calculation involving discounts. The model has no way to distinguish data quality problems from real patterns — that judgment is yours.

2.2 Fixing Data Types

Wrong data types are the most common wrangling problem and the easiest to miss. `df.info()` shows both the type and the non-null count — run it before anything else.

Common Type Problems

Column	Wrong Type	Correct Type	Cause
`order_date`	`object`	`datetime64`	Dates stored as strings
`revenue`	`object`	`float64`	Currency symbols or commas in source file
`customer_id`	`float64`	`Int64`	Pandas converts int columns to float when NaNs are present
`is_returned`	`int64`	`bool`	Stored as 0/1 instead of True/False

Converting Types

```
# Dates stored as strings
df['order_date'] = pd.to_datetime(df['order_date'])

# Numbers stored as strings (e.g., "$1,250.00")
df['revenue'] = df['revenue'].str.replace('[$,]', '', regex=True)
df['revenue'] = pd.to_numeric(df['revenue'], errors='coerce')
# errors='coerce' turns any value that cannot be parsed into NaN
# rather than raising an error — check for new nulls afterward

# Integer IDs stored as float
df['customer_id'] = df['customer_id'].astype('Int64')
# Capital-I Int64 is pandas nullable integer — handles NaN without converting to float

# Categorical columns with few unique values
df['region'] = df['region'].astype('category')
```

Extracting Features from Dates

Raw datetime columns are not passed to models — they are converted into numeric features.

```
df['order_month']     = df['order_date'].dt.month
df['order_dayofweek'] = df['order_date'].dt.dayofweek  # 0 = Monday
df['order_quarter']   = df['order_date'].dt.quarter
df['days_since_start'] = (df['order_date'] - df['order_date'].min()).dt.days
```

🤖 ML Connection — Types and scikit-learn

scikit-learn will raise a `ValueError` if you pass a column with dtype `object`. It cannot convert strings to numbers on its own. Every column fed to a model must be numeric — either genuinely numeric, encoded from a category (Week 3), or extracted from a date. Fixing types in Week 2 prevents runtime errors in Weeks 4–7.

2.3 Missing Data

Detecting Missing Values

```
# Count of nulls per column
df.isnull().sum()

# As a percentage of total rows
(df.isnull().sum() / len(df) * 100).round(1)

# Rows where any column is null
df[df.isnull().any(axis=1)]
```

A null rate above 40% in a column is a signal to investigate before deciding how to handle it. A null rate that high may mean the column was not collected consistently — and a column that is missing half its values is unlikely to be a reliable predictor.

Why Data Goes Missing

Missing data is not random noise. Understanding why a value is missing changes how you handle it.

Pattern	Description	Example
Missing at random	Missingness is unrelated to the value itself	A sensor occasionally fails to record
Missing not at random	Missingness is related to the value	High-value customers opt out of revenue tracking
Structurally missing	The value does not apply	A discount field is null because no discount was given

Structurally missing values are often best filled with zero or a sentinel value. Missing-not-at-random is the most dangerous — imputing these values can hide a real pattern.

Handling Strategies

```
# Drop rows with nulls in specific columns
df.dropna(subset=['revenue'], inplace=True)

# Fill numeric nulls with the median
df['revenue'] = df['revenue'].fillna(df['revenue'].median())

# Fill numeric nulls with the mean
df['revenue'] = df['revenue'].fillna(df['revenue'].mean())

# Fill categorical nulls with the mode
df['region'] = df['region'].fillna(df['region'].mode()[0])

# Fill with a fixed value
df['discount_pct'] = df['discount_pct'].fillna(0)

# Flag missingness as a separate binary feature, then fill
df['revenue_missing'] = df['revenue'].isnull().astype(int)
df['revenue'] = df['revenue'].fillna(df['revenue'].median())
```

Use median over mean when a column is skewed — a few very large values pull the mean upward, and filling missing values with an inflated mean misrepresents the typical case. Use mode for categorical columns. Use zero only when zero is a meaningful value (e.g., no discount was applied).

💼 Business Context — The Cost of Getting This Wrong

Filling a missing revenue value with zero tells the model that transaction had zero revenue. If the value is missing because the data was not collected — not because nothing was sold — that zero is a fabrication. The model will learn that whatever other characteristics that row has are associated with zero revenue. That learned pattern is wrong. The flag-and-fill approach is safer: it preserves the imputed value for modelling while giving the model a signal that the original was unknown.

2.4 Invalid Values

Invalid values are present but wrong. They pass a null check, so `isnull().sum()` will not find them.

Common Forms

Problem	Example	Detection
Out-of-range numeric	`discount_pct = 1.75`	`df[df['discount_pct'] > 1]`
Negative in a non-negative field	`units_sold = -3`	`df[df['units_sold'] < 0]`
Impossible combination	`units_sold = 0` but `revenue = 850`	`df[(df['units_sold'] == 0) & (df['revenue'] > 0)]`
Inconsistent categories	`'northeast'` and `'Northeast'` in same column	`df['region'].value_counts()`

Fixing Invalid Values

```
# Cap discount at 1.0 (100%)
df.loc[df['discount_pct'] > 1.0, 'discount_pct'] = 1.0

# Remove rows with negative units
df = df[df['units_sold'] >= 0]

# Standardize category strings
df['region'] = df['region'].str.strip().str.title()
# .strip() removes leading/trailing whitespace
# .title() capitalises first letter of each word
```

Duplicates

```
# Count duplicate rows
df.duplicated().sum()

# View duplicate rows
df[df.duplicated(keep=False)]

# Drop duplicates, keep first occurrence
df.drop_duplicates(inplace=True)

# Duplicates based on key columns only
df.drop_duplicates(subset=['transaction_id'], inplace=True)
```

Full row duplicates usually indicate a data pipeline error — the same event was written to the database twice. Key-based duplicates (same transaction ID, different values) are more serious and require investigation before dropping.

2.5 Feature Identification

A feature is a column you will use as an input to a model. Not every column in a dataset is a useful feature. Identifying which columns to keep, transform, or drop is a judgment call — it requires both data knowledge and domain knowledge.

Columns to Drop

Reason to Drop	Example	Why
Identifier	`transaction_id`, `customer_id`	Unique per row — no predictive signal
Free text	`customer_notes`	Requires NLP; out of scope for tabular models
High null rate (>40%)	`secondary_phone`	Too sparse to be reliable
Near-constant	`country` if 99% is `'Canada'`	No variance means no signal
Leakage	`days_to_resolution` when predicting resolution	Future information — would not be available at prediction time
Post-outcome	`refund_amount` when predicting churn	Recorded after the event you are predicting

```
# Check unique values per column
df.nunique()

# Identify near-constant columns (less than 2 unique values)
low_variance = [col for col in df.columns if df[col].nunique() < 2]

# Drop columns
df.drop(columns=['transaction_id', 'customer_notes'], inplace=True)
```

Data Leakage

Leakage is the most serious feature identification mistake. It occurs when a feature contains information that would not be available at the time a real prediction is made. A model with leakage will perform excellently on test data and fail in production — because in production, the future information does not exist yet.

If you are predicting whether a customer will churn next month, you cannot use `total_calls_to_support_this_month` as a feature — you do not have that value until the month ends. You can use `avg_calls_to_support_last_3_months` — that is available before the prediction window.

🤖 ML Connection — Features Determine What the Model Can Learn

A model can only find patterns in the features you give it. If the relevant information is in a column you dropped, the model cannot compensate for its absence. If a leakage column is included, the model will appear to perform well but will fail in deployment. Feature identification is where business understanding matters most — no algorithm can tell you what information would actually be available when a prediction needs to be made.

2.6 The Wrangling Checklist

Run through this sequence on every new dataset before moving to modelling.

```
# ── Step 1: Shape and types ──────────────────────────────────────────
print(df.shape)
print(df.dtypes)
df.info()

# ── Step 2: Missing values ───────────────────────────────────────────
print(df.isnull().sum())
print((df.isnull().sum() / len(df) * 100).round(1))

# ── Step 3: Duplicates ───────────────────────────────────────────────
print(df.duplicated().sum())

# ── Step 4: Value distributions — spot invalid values ────────────────
df.describe()
for col in df.select_dtypes(include='object').columns:
    print(df[col].value_counts())

# ── Step 5: Feature identification ───────────────────────────────────
print(df.nunique())
# Drop identifiers, constants, high-null columns, and leakage
```

This is not a one-pass process. Each step will surface issues that affect earlier steps. Run it, fix what you find, and run it again until the output looks clean.

2.7 Chapter Summary

Concept	Key Point
Data wrangling	Transform raw data into a clean, model-ready form before any analysis
`df.info()`	Start here — shows types and null counts in one view
`pd.to_datetime()`	Converts string dates to datetime — extract month, day, quarter afterward
`pd.to_numeric(errors='coerce')`	Converts strings to numbers, turns unparseable values into NaN
Missing at random vs. not at random	Why the data is missing changes how you handle it
`fillna(median)`	Preferred for skewed numeric columns — mean is pulled by outliers
Flag-and-fill	Create a binary missing indicator before imputing — preserves information
Invalid values	Present but wrong — use `df[condition]` to detect, `.loc[]` to fix
`str.strip().str.title()`	Standardize inconsistent category strings
`drop_duplicates()`	Remove exact row duplicates — investigate key-based duplicates
Data leakage	Feature contains future information — causes inflated test scores and production failure
Feature identification	Drop IDs, constants, high-null columns, and leakage before modelling

Review Questions

1. A dataset has a column `signup_date` with dtype `object`. After converting it to datetime, what three numeric features would you extract from it that might be useful for predicting customer churn? Explain why each could be predictive.

2. You are cleaning a dataset of insurance claims. The column `claim_amount` has 22% missing values. Describe two different strategies for handling this missingness and explain what assumption each strategy makes about why the data is missing.
3. You build a model to predict whether a sales call will result in a closed deal. Your feature set includes `call_duration_minutes`, `rep_id`, `lead_score`, and `deal_value`. Identify any potential leakage and explain why it is a problem.
4. A `region` column contains the values `'northeast'`, `'Northeast'`, `'NORTHEAST'`, and `'North East'`. Write the pandas code to standardize all of these to `'Northeast'` in a single operation, without hardcoding a replacement for each variant.
5. After running `df.describe()` on a customer dataset, you notice the `age` column has a minimum value of -4 and a maximum of 142. What does this tell you, and what are two ways you could handle the invalid values?

Chapter 3 — Feature Engineering and the ML Pipeline

Data is clean. Features are identified. Before training a model, three things remain: encode categorical columns as numbers, scale numeric columns to a common range, and split the data into training and test sets. This chapter covers all three — and the rule that governs the order in which they must happen.

3.1 What Is Feature Engineering?

Feature engineering transforms cleaned columns into a form a model can use effectively. Most machine learning algorithms require numeric input. They also perform better when numeric features operate on a similar scale. And they must be evaluated on data they have never seen during training.

Feature engineering addresses all three requirements:

Task	What It Solves
Encoding	Converts categorical columns to numbers
Scaling	Brings numeric columns to a comparable range
Train/test split	Ensures evaluation happens on unseen data

The order matters. Split first, then encode and scale — and encode and scale using only the training data. This chapter explains why.

3.2 Encoding Categorical Variables

Machine learning models cannot process strings. A column containing `'Northeast'`, `'West'`, `'South'`, and `'Midwest'` must be converted to numbers before it can be used as a feature.

One-Hot Encoding

One-hot encoding creates a new binary column for each category. A row gets a 1 in the column for its category and 0 in all others.

```
region         →    region_Northeast  region_South  region_West
Northeast           1                 0             0
West                0                 0             1
South               0                 1             0
Midwest             0                 0             0   ← implied by all zeros
```

With pandas `get_dummies`:

```
df_encoded = pd.get_dummies(df, columns=['region', 'product'], drop_first=False)
```

With scikit-learn `OneHotEncoder` (required inside a pipeline):

```
from sklearn.preprocessing import OneHotEncoder


ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoded = ohe.fit_transform(df[['region', 'product']])
```

`handle_unknown='ignore'` tells the encoder to output all zeros for any category it has never seen — rather than raising an error. This matters in production, where new category values may appear.

The Dummy Variable Trap

With k categories, only k-1 columns are needed. The dropped column is implied: if all others are 0, the observation belongs to the dropped category. Including all k columns creates perfect multicollinearity and causes problems for linear models.

```
# drop_first=True drops one column per categorical variable
df_encoded = pd.get_dummies(df, columns=['region'], drop_first=True)
```

`drop_first` is good practice for linear models (regression, logistic regression). Tree-based models (random forests, gradient boosting) are not affected — but the habit is worth keeping.

Ordinal Encoding

Use ordinal encoding when categories have a meaningful order.

```
from sklearn.preprocessing import OrdinalEncoder


# Low < Medium < High — the order matters
oe = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
df['priority_encoded'] = oe.fit_transform(df[['priority']])
# Low → 0, Medium → 1, High → 2
```

Never use ordinal encoding on nominal variables. Encoding `'Northeast'`, `'West'`, `'South'` as 0, 1, 2 implies `West` is twice `Northeast` — a relationship that does not exist.

💼 Business Context — Encoding Choices Have Consequences

A region column encoded as 0, 1, 2, 3 tells a linear model that Midwest (3) is three times Northeast (0). The model will use that arithmetic relationship in its predictions. The predictions will be wrong, and the error will be invisible — the model will train and score without complaint. One-hot encoding avoids this entirely by making each region independent of the others.

3.3 Scaling Numeric Features

Many algorithms measure distances or apply weights to features. When one feature ranges from 0 to 1 (discount rate) and another ranges from 0 to 100,000 (annual revenue), the larger-scale feature dominates. Scaling brings all numeric features to a comparable range.

StandardScaler

Transforms each feature to have mean = 0 and standard deviation = 1.

$$z = \frac{x - \bar{x}}{s}$$

```
from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

After scaling, a value of 1.5 means "1.5 standard deviations above the mean for this feature." All features are now on the same footing regardless of their original units.

MinMaxScaler

Transforms each feature to the range [0, 1].

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
from sklearn.preprocessing import MinMaxScaler


scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

MinMaxScaler is sensitive to outliers — one extreme value compresses everything else toward 0 or 1. StandardScaler is more robust in most business datasets. Use MinMaxScaler when you need values bounded to [0, 1], such as for neural network inputs.

Which Algorithms Need Scaling

Algorithm	Needs Scaling?	Why
Linear regression	Yes	Coefficients are directly affected by feature scale
Logistic regression	Yes	Same reason
K-Nearest Neighbors	Yes	Distance-based — scale dominates
Support Vector Machine	Yes	Margin calculation is scale-dependent
Decision tree	No	Splits on thresholds, not distances
Random forest	No	Ensemble of trees — scale-invariant
Gradient boosting	No	Same

Tree-based models do not require scaling. Applying it does not hurt them, but it adds no benefit. In Weeks 4 and 5 you will use both linear and tree-based models — apply scaling by default and remove it only if you have a specific reason.

🤖 ML Connection — Scaling Is a Learned Transformation

StandardScaler learns the mean and standard deviation from the training data. That is why you call `.fit_transform()` on training data and `.transform()` on test data — the test set is scaled using the training set's statistics, not its own. If you fit the scaler on the full dataset before splitting, the test set's statistics influence the training transformation. This is data leakage, and it is the topic of the next section.

3.4 The Train/Test Split

A model evaluated on its own training data will appear to perform better than it actually is — it has already seen those examples during training. The test set must contain data the model has never encountered.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% held out for testing
    random_state=42,    # reproducible split
    stratify=y          # preserves class proportions — use for classification
)

print(f"Training: {X_train.shape[0]} rows | Test: {X_test.shape[0]} rows")
```

Stratified Splitting

For classification problems, use `stratify=y`. Without it, a random split might put 95% of the minority class in training and 5% in the test set — making evaluation unreliable.

```
# Without stratify: class proportions may differ between train and test
# With stratify=y: both sets have the same proportion of each class
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Choosing the Split Ratio

Split	Use When
80/20	Standard — sufficient data in both sets
70/30	Smaller dataset — more data in test improves evaluation reliability
90/10	Very small dataset — maximizes training data

For most datasets in this course, 80/20 is appropriate.

3.5 The Critical Rule: Fit Only on Training Data

This is the most important rule in this chapter. Encoders and scalers must be **fit on the training set only**, then applied to both training and test sets.

```
# ── WRONG — data leakage ──────────────────────────────────────────
scaler.fit(X)                              # learns from all data, including test
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# ── CORRECT ───────────────────────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler.fit(X_train)                        # learns only from training data
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test) # applies training statistics to test
```

When the scaler is fit on all data, the test set's mean and variance influence the transformation — the model indirectly "sees" the test set during training. The evaluation score looks better than it should. In production, where future data has no prior statistics to contribute, performance drops.

The same rule applies to encoders: fit `OneHotEncoder` on training data only. New categories in the test set are handled by `handle_unknown='ignore'`.

🤖 ML Connection — Why This Error Is Hard to Catch

Leakage from fitting preprocessors on all data is subtle. The model trains, evaluates, and produces results that look reasonable. Nothing crashes. The error only becomes visible when the deployed model underperforms relative to evaluation — which is often weeks or months after deployment. Fitting on training data only is non-negotiable. The Pipeline object (Week 7) enforces this automatically: it re-fits transformers on each training fold and applies them to each validation fold without any manual management.

3.6 Putting It Together: ColumnTransformer

Real datasets have a mix of numeric and categorical columns that require different transformations. `ColumnTransformer` applies different preprocessing steps to different columns in one object.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

categorical_cols = ['region', 'product']
numerical_cols   = ['units_sold', 'unit_price', 'discount_pct', 'order_month']

preprocessor = ColumnTransformer(transformers=[
    ('cat', OneHotEncoder(handle_unknown='ignore', drop='first'), categorical_cols),
    ('num', StandardScaler(), numerical_cols),
])

# Fit on training data, transform both sets
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed  = preprocessor.transform(X_test)

print(f"Processed training shape: {X_train_processed.shape}")
```

The output of `ColumnTransformer` is a numpy array ready to pass directly to a scikit-learn model. In Week 7, this preprocessor is packaged inside a `Pipeline` with the model itself — so fit, transform, and predict all happen in one `.fit()` call.

💼 Business Context — Preprocessing Is Part of the Model

When you deploy a model, you deploy the entire preprocessing chain with it. A customer record arrives as raw data — region strings, float prices, integer unit counts. The deployed system must apply the exact same transformations that were applied during training before the model makes a prediction. If the preprocessor is not packaged with the model, someone has to manually reapply it in production — a source of silent errors. `ColumnTransformer` and `Pipeline` (Week 7) solve this by keeping preprocessing and modelling in one object.

3.7 Chapter Summary

Concept	Key Point
Feature engineering	Encode, scale, and split — in that order within the split
One-hot encoding	Creates k-1 binary columns for a nominal variable with k categories
`handle_unknown='ignore'`	Outputs all zeros for unseen categories in production — always use it
Ordinal encoding	Maps ordered categories to integers — do not use on nominal variables
The dummy variable trap	Including all k columns causes multicollinearity — drop one
StandardScaler	Mean = 0, std = 1 — robust default for most algorithms
MinMaxScaler	Range [0, 1] — sensitive to outliers
Tree models and scaling	Decision trees and random forests do not require scaling
`train_test_split`	`test_size=0.2`, `random_state=42`, `stratify=y` for classification
Fit only on training data	Fit encoders and scalers on X_train only, transform both sets separately
`ColumnTransformer`	Applies different preprocessing to different column types in one object

Review Questions

1. A dataset has a `priority` column with values `'Low'`, `'Medium'`, and `'High'`. A colleague one-hot encodes it. Explain why this is the wrong encoding choice and what should be used instead.

2. You are building a model to predict customer lifetime value (a continuous dollar amount). Your numeric features include `annual_income` (range: $25,000–$250,000) and `num_purchases` (range: 1–15). Explain what problem this scale difference causes for a linear regression model and how StandardScaler fixes it.

3. Write the correct sequence of code to: split `X` and `y` into 80/20 training and test sets with `random_state=42`, fit a `StandardScaler` correctly, and transform both sets. Then show the incorrect version and explain what the mistake causes.

4. A `ColumnTransformer` is configured with `OneHotEncoder` on the `region` column and `StandardScaler` on the `revenue` column. The encoder was fit on training data that contains only `'Northeast'`, `'West'`, and `'South'`. During test evaluation, a row with region `'Midwest'` appears. What happens, and why is this the desired behaviour?
5. Explain in plain language why fitting a `StandardScaler` on the full dataset before splitting — rather than on the training set only — is a form of data leakage. What specifically leaks, and how does it affect evaluation scores?

Chapter 4 — Supervised Learning: Regression

Regression answers one type of question: *how much?* How much will this property sell for? How much revenue will this campaign generate? How much will this customer spend over the next 12 months? The target is always a continuous numeric value — not a category, not a label. This chapter builds a complete regression workflow: prepare the data, fit the model, evaluate it with three complementary metrics, and interpret what the model learned.

4.1 Regression Problems

A regression problem has a continuous numeric target. The model learns a function that maps input features to a predicted number.

Target	Example Features	Business Use
Property sale price	Square footage, neighbourhood, age, bedrooms	Automated valuation for mortgage underwriting
Monthly revenue	Ad spend, channel, season, campaign duration	Budget allocation and forecasting
Employee salary	Years of experience, education, department	Compensation benchmarking
Customer lifetime value	Purchase frequency, average order value, tenure	Retention investment decisions

Regression is the right tool when: (1) the target is numeric and continuous, and (2) the business decision requires knowing the predicted amount, not just a category.

Regression vs. Classification

Both are supervised learning — both use labelled historical data. The distinction is the target type:

- **Regression:** the target is a number (`sale_price = $485,000`)
- **Classification:** the target is a category (`will_sell = Yes`)

You can convert any regression problem into a classification problem by binning the target (e.g., "above or below the median"). The reverse is also true — logistic regression is a classification model despite the name. Choose based on what the business needs. If the decision requires knowing the exact predicted value, use regression. If it only requires a yes/no or a category, use classification.

4.2 Linear Regression — The Model

Linear regression fits a straight-line (hyperplane) relationship between features and the target. The model equation is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

- $\hat{y}$ — the predicted value
- β_0 — the intercept: the predicted value when all features equal zero
- $\beta_1 \ldots \beta_n$ — coefficients: the change in the prediction for a one-unit increase in each feature, holding all others constant
- $x_1 \ldots x_n$ — the feature values for a given row

What the Model Learns

Training finds the set of coefficients that minimizes the sum of squared differences between predicted values and actual values — the **ordinary least squares** (OLS) objective. Scikit-learn handles this automatically. What you need to understand is the result: each coefficient captures the model's best estimate of how much one unit of that feature is worth.

Example: A property price model fit on sale data might produce:

Feature	Coefficient	Interpretation
sqft	235.40	Each additional square foot adds $235 to predicted price
bedrooms	12,800	Each additional bedroom adds $12,800
age_years	−1,620	Each additional year of age reduces price by $1,620
has_garage	22,500	A garage adds $22,500
neighbourhood_Downtown	88,400	Downtown adds $88,400 vs. the baseline neighbourhood

The intercept (β_0) is the predicted price when all feature values are zero. It rarely has a useful literal interpretation — it anchors the regression line to the data.

When Linear Regression Is Appropriate

Linear regression assumes the relationship between features and target is approximately linear. It also assumes the error (the gap between predicted and actual values) is random, not systematic. Chapter 4.5 covers how to check these assumptions.

Linear regression works well when: the target is continuous and unbounded, the relationships are roughly linear, and you need an interpretable model whose coefficients explain the prediction. It does not work well when the relationships are highly nonlinear or when interactions between features dominate the signal — those cases call for tree-based models (Chapter 6).

4.3 Fitting a Linear Regression Model in Scikit-Learn

Scikit-learn's `LinearRegression` follows the same fit/predict/score API you will use for every model in this course.

The Workflow

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Preprocess — reuse the ColumnTransformer from Week 3
categorical_cols = ['neighbourhood', 'property_type']
numerical_cols   = ['sqft', 'bedrooms', 'bathrooms', 'age_years', 'has_garage']

preprocessor = ColumnTransformer(transformers=[
    ('cat', OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=False),
categorical_cols),
    ('num', StandardScaler(), numerical_cols),
])

# 2. Split — always before fitting anything
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Fit the preprocessor on training data only
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed  = preprocessor.transform(X_test)

# 4. Fit the model
model = LinearRegression()
model.fit(X_train_processed, y_train)

# 5. Predict
y_pred_train = model.predict(X_train_processed)
y_pred_test  = model.predict(X_test_processed)
```

Accessing Coefficients

```
# Retrieve coefficient names and values
feature_names = preprocessor.get_feature_names_out()
coef_df = pd.DataFrame({
    'feature':     feature_names,
    'coefficient': model.coef_
}).sort_values('coefficient', ascending=False)

print(coef_df)
print(f"\nIntercept: {model.intercept_:,.0f}")
```

The coefficients from a scaled model reflect the effect of a one-standard-deviation change in each feature, not a one-unit change. This makes the coefficients comparable across features — a feature with a large coefficient is genuinely more influential, not just measured on a larger scale.

4.4 Evaluation Metrics

Three metrics evaluate regression models. Use all three — each catches different problems.

Mean Absolute Error (MAE)

MAE is the average absolute difference between predicted and actual values.

```
MAE = (1/n) × Σ |yᵢ − ŷᵢ|
```

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred_test)
print(f"MAE: ${mae:,.0f}")
# MAE: $28,400
```

Interpretation: On average, the model's predictions are off by $28,400. This is expressed in the same units as the target, which makes it easy to communicate to a non-technical stakeholder. If the typical property sells for $450,000, a $28,400 MAE is about 6% average error.

Limitation: MAE treats all errors equally. A $10,000 error and a $100,000 error each contribute proportionally to the average.

Root Mean Squared Error (RMSE)

RMSE squares each error before averaging, then takes the square root. Squaring makes large errors count more than small ones.

```
RMSE = √[ (1/n) × Σ (yᵢ − ŷᵢ)² ]
```

```
from sklearn.metrics import root_mean_squared_error


rmse = root_mean_squared_error(y_test, y_pred_test)
print(f"RMSE: ${rmse:,.0f}")
# RMSE: $41,200
```

Interpretation: RMSE is in the same units as the target. It is always ≥ MAE. The gap between MAE and RMSE reveals how much the model struggles with large errors — a large gap means the model has a few very wrong predictions pulling RMSE up. If MAE and RMSE are close, the errors are consistent in size.

When to prefer RMSE: When large errors are especially costly — a model that's occasionally off by $150,000 on a property valuation is worse than one that's consistently off by $30,000, even if the average is similar.

R^2 (R-Squared)

R^2 measures what proportion of the variance in the target the model explains, on a 0–1 scale.

```
from sklearn.metrics import r2_score


r2 = r2_score(y_test, y_pred_test)
print(f"R²: {r2:.3f}")
# R²: 0.847
```

Interpretation: An R^2 of 0.847 means the model explains 84.7% of the variation in sale prices. The remaining 15.3% is unexplained — driven by features the model does not have (condition, interior finish quality, negotiation) or by irreducible randomness.

Interpretation benchmarks:

R^2	Rough interpretation
> 0.90	Strong — appropriate for many business applications
0.70–0.90	Moderate — useful for ranking and directional decisions
0.50–0.70	Weak — better than random, but predictions carry significant uncertainty
< 0.50	Poor — the model explains less than half the variation

These benchmarks depend heavily on domain. R^2 of 0.60 is strong for predicting human behavior; it is poor for predicting physical measurement error.

Train vs. Test Metrics — Detecting Overfitting

Always compute metrics on both training and test sets.

```
print(f"Train R²: {r2_score(y_train, y_pred_train):.3f}")
print(f"Test R²:  {r2_score(y_test,  y_pred_test):.3f}")
```

Train R^2	Test R^2	Diagnosis
0.85	0.84	Healthy — generalizes well
0.95	0.72	Overfit — model memorized training data
0.61	0.60	Underfit — model is too simple for the data

Linear regression rarely overfits severely — it is a low-variance model. Overfitting becomes a much bigger concern with decision trees and random forests (Chapter 6).

4.5 Interpreting Coefficients

Coefficients are the model's learned relationships. Reading them carefully is part of the job.

Scaled vs. Unscaled Coefficients

When features are scaled with `StandardScaler`, all coefficients are expressed in units of standard deviations. This makes them directly comparable — a coefficient of 85,000 on `sqft` and a coefficient of 12,000 on `bedrooms` both mean "one standard deviation increase in this feature adds that many dollars to the predicted price."

Without scaling, `sqft` values are in the hundreds (400–3,500) while `has_garage` is 0 or 1. Their raw coefficients are not comparable — `sqft` gets a small coefficient simply because its unit is tiny. Scale before you interpret.

Building a Coefficient Table

```
coef_df = pd.DataFrame({
    'feature':     preprocessor.get_feature_names_out(),
    'coefficient': model.coef_
}).sort_values('coefficient', key=abs, ascending=False)

print(coef_df.to_string(index=False))
```

Sorting by absolute value of the coefficient identifies the most influential features — the variables the model is using most heavily to make its predictions.

What Coefficients Cannot Tell You

A large coefficient does not mean the feature *causes* the outcome. A downtown neighbourhood flag might have a large positive coefficient because downtown properties are bigger, newer, and have garages — all correlated features. The coefficient captures the partial effect after accounting for other features, but correlation in the data can still confound interpretation.

Use coefficient tables for operational insight — "the model relies most heavily on square footage and neighbourhood" — not for causal claims.

4.6 Residual Analysis

A **residual** is the difference between the actual value and the predicted value: `residual = y − ŷ`. Residual analysis checks whether the model's errors are random (good) or systematic (bad).

```
import matplotlib.pyplot as plt

residuals = y_test - y_pred_test

# Plot residuals vs. predicted values
plt.figure(figsize=(8, 4))
plt.scatter(y_pred_test, residuals, alpha=0.5)
plt.axhline(0, color='red', linewidth=1)
plt.xlabel('Predicted Sale Price')
plt.ylabel('Residual')
plt.title('Residuals vs. Predicted Values')
plt.tight_layout()
plt.show()
```

What to Look For

Pattern in residual plot	Diagnosis
Random scatter around zero	Healthy — errors are random
Funnel shape (spread increases with predicted value)	Heteroscedasticity — variance is not constant; consider log-transforming the target
Curve or arch	Nonlinear relationship — the model is too simple for the data
Systematic band above or below zero	Bias — the model consistently over- or under-predicts for certain values

A random scatter centered at zero means the model has captured the major patterns and the remaining error is noise. A systematic pattern means there is signal left in the data that the model has not learned.

Distribution of Residuals

```
plt.figure(figsize=(6, 4))
plt.hist(residuals, bins=30, edgecolor='white')
plt.axvline(0, color='red', linewidth=1)
plt.xlabel('Residual')
plt.title('Residual Distribution')
plt.tight_layout()
plt.show()
```

Residuals should be approximately normally distributed and centered at zero. A long right tail suggests the model underestimates prices for the most expensive properties — a common pattern in real estate data where luxury properties behave differently from the rest.

Chapter 4 Summary

Concept	Key Takeaway
Regression target	A continuous numeric value — how much, not which category
Linear regression equation	$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$ — each coefficient is the effect of one unit of that feature
Fit/predict/score API	`.fit()` on training data, `.predict()` on both sets, evaluate on test set
`.coef_` and `.intercept_`	Stored after fitting — access the learned coefficients and baseline prediction
MAE	Average absolute error — same units as target, easy to communicate
RMSE	Penalizes large errors more — always ≥ MAE; gap between them reveals outlier errors
R^2	Proportion of variance explained — 1.0 is perfect, 0.0 means the model predicts the mean
Train vs. test metrics	Compare both — a large gap signals overfitting
Coefficient interpretation	Scaled coefficients are comparable across features; large absolute value = more influential
Residual analysis	Random scatter around zero = healthy; systematic patterns = model limitation
Causation warning	Coefficients show correlation, not causation — communicate accordingly

Review Questions

1. A property dataset has two features: `sqft` (range 400–3,500) and `has_garage` (0 or 1). After fitting a linear regression without scaling, the coefficient for `sqft` is 48.3 and the coefficient for `has_garage` is 22,500. A colleague says the garage is clearly more important than square footage because 22,500 >> 48.3. What is wrong with this reasoning, and how would you correct it?

2. Your regression model has Train $R^2 = 0.91$ and Test $R^2 = 0.63$. What does this tell you about the model? Name two actions you could take to address the problem.

3. You compute MAE = $18,000 and RMSE = $47,000 on the same test set. What does the large gap between these two metrics tell you about the model's errors? How would you investigate further?
4. Write the scikit-learn code to fit a `LinearRegression` model on preprocessed training data and print the MAE and R^2 on the test set. Assume `X_train_processed`, `X_test_processed`, `y_train`, and `y_test` are already defined.
5. A residual plot shows a funnel shape — residuals are small and tightly clustered for low predicted values, but spread out widely for high predicted values. What is this pattern called, and what does it suggest about the model?

Chapter 5 — Supervised Learning: Classification

Regression predicts *how much*. Classification predicts *which one*. Will this customer churn or stay? Is this transaction fraudulent or legitimate? Will this loan default? The target is a category — and the model's job is to assign each new observation to the right one. This chapter covers two classification algorithms, a complete set of evaluation metrics, and the business judgment required to choose between them.

5.1 Classification Problems

A classification problem has a categorical target. The most common case in business is **binary classification**: two possible outcomes, typically encoded as 0 (negative) and 1 (positive).

Target	Example Features	Business Use
Churned (yes/no)	Tenure, contract type, monthly charges	Retention — identify at-risk customers before they leave
Defaulted (yes/no)	Credit score, income, debt-to-income ratio	Credit risk — decide whether to approve a loan
Fraudulent (yes/no)	Transaction amount, location, time, device	Fraud detection — flag transactions for review
Converted (yes/no)	Campaign channel, email opens, visit count	Marketing — target high-probability converters

Multi-class classification — three or more categories — exists (e.g., predicting which of five products a customer will buy next), but binary classification covers the majority of business use cases and is the foundation for understanding multi-class methods.

Class Balance

Classification problems often have **imbalanced classes** — far more observations in one category than the other. Fraud detection is extreme: 99.9% legitimate, 0.1% fraudulent. Customer churn is more moderate: typically 10–30% churn rate. Class imbalance is not a problem to fix before modelling — it is a reality to account for in evaluation. The accuracy metric fails on imbalanced data. Section 5.4 explains why and what to use instead.

5.2 Logistic Regression

Despite the name, logistic regression is a classification model. It predicts the **probability** that an observation belongs to the positive class, then classifies based on a threshold (default: 0.5).

The Model

Logistic regression applies the sigmoid function to a linear combination of features:

```
P(y = 1) = 1 / (1 + e^(−(β₀ + β₁x₁ + ... + βₙxₙ)))
```

The sigmoid function maps any real number to a value between 0 and 1 — a probability. Features still have coefficients (as in linear regression), but the output is always a valid probability, not an unbounded number.

Fitting and Predicting

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_processed, y_train)

# Predict classes (0 or 1)
y_pred = model.predict(X_test_processed)

# Predict probabilities
y_proba = model.predict_proba(X_test_processed)
# y_proba has two columns: [:, 0] = P(y=0), [:, 1] = P(y=1)
print(y_proba[:5])
# [[0.72, 0.28],
#  [0.18, 0.82],
#  [0.91, 0.09], ...]
```

`max_iter=1000` increases the solver's iteration limit — the default (100) is often too low for datasets with many features and causes a convergence warning.

What Logistic Regression Is Good At

Logistic regression is fast, interpretable, and works well when the relationship between features and the log-odds of the outcome is approximately linear. It is the first model to try for any binary classification problem. Its coefficients are interpretable (positive coefficient = feature increases probability of the positive class), and it provides calibrated probability estimates — the predicted 0.7 probability means "roughly 70% of similar observations are positive."

5.3 Decision Trees

A decision tree splits the data into branches based on feature values, creating a series of if-then rules that lead to a prediction. Each internal node is a split condition; each leaf is a predicted class.

How Trees Split

At each node, the tree finds the feature and threshold that best separates the classes in the training data. "Best" means the split produces the purest child nodes — measured by **Gini impurity** (the default in scikit-learn) or entropy.

```
Gini impurity = 1 − Σ(pᵢ²)
```

A node with all one class has impurity = 0 (pure). A 50/50 split has impurity = 0.5 (maximally impure). The tree greedily selects the split that reduces impurity the most at each step.

Fitting a Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train_processed, y_train)

y_pred_tree = tree.predict(X_test_processed)
```

`max_depth` is the most important hyperparameter. Without a limit, the tree will grow until every training observation is in its own leaf — perfect training accuracy, near-zero test accuracy. This is extreme overfitting. `max_depth=4` is a reasonable starting point for exploration; the right value is found through cross-validation (Chapter 6).

Feature Importance

Decision trees provide **feature importance** scores — the fraction of impurity reduction each feature contributed across all splits. This is a native output, not derived from coefficients.

```
importances = pd.DataFrame({
    'feature':    preprocessor.get_feature_names_out(),
    'importance': tree.feature_importances_
}).sort_values('importance', ascending=False)

print(importances)
```

Feature importances sum to 1. A feature with importance 0.35 contributed 35% of the total impurity reduction. Features the tree never used have importance 0.

When to Use Decision Trees

Decision trees are interpretable — you can print the tree and trace any prediction from root to leaf. They handle nonlinear relationships and interactions between features without any preprocessing. They do not require feature scaling.

Their weakness is instability: a small change in training data can produce a very different tree. They also overfit easily when `max_depth` is unconstrained. Random forests (Chapter 6) address both weaknesses by building many trees and averaging their predictions.

5.4 The Confusion Matrix

Accuracy — the fraction of predictions that are correct — is the wrong primary metric for most classification problems. On a dataset where 90% of observations are negative, a model that predicts "negative" for every observation achieves 90% accuracy without learning anything.

The **confusion matrix** breaks predictions into four categories:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- **True Positive (TP):** correctly predicted positive (predicted churn, actually churned)
- **True Negative (TN):** correctly predicted negative (predicted retained, actually retained)
- **False Positive (FP):** predicted positive, actually negative (predicted churn, actually stayed) — a false alarm
- **False Negative (FN):** predicted negative, actually positive (predicted retained, actually churned) — a miss

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

Accuracy = (TP + TN) / (TP + TN + FP + FN). On an imbalanced dataset, accuracy tells you almost nothing about how well the model finds the positive class.

5.5 Precision, Recall, and F1

Three metrics complement the confusion matrix:

Precision

Of all observations the model predicted as positive, what fraction actually were?

```
Precision = TP / (TP + FP)
```

High precision means few false alarms. A precision of 0.85 means 85% of customers flagged as likely churners actually churned.

Use precision when false positives are costly. A false alarm in fraud detection means freezing a legitimate customer's card — that has a real cost.

Recall (Sensitivity)

Of all observations that are actually positive, what fraction did the model find?

```
Recall = TP / (TP + FN)
```

High recall means few misses. A recall of 0.72 means the model caught 72% of actual churners and missed 28%.

Use recall when false negatives are costly. Missing a churner who then leaves is expensive — the model failed to trigger a retention action that could have kept them.

F1 Score

F1 is the harmonic mean of precision and recall. It is a single number that balances both:

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

F1 is high only when both precision and recall are high. It penalizes extreme imbalance between the two — a model with 0.95 precision and 0.10 recall will have a low F1, reflecting that it catches almost nothing.

The Classification Report

```
from sklearn.metrics import classification_report


print(classification_report(y_test, y_pred, target_names=['Retained', 'Churned']))
```

Output:

```
              precision    recall  f1-score   support


    Retained       0.87      0.92      0.89        73
     Churned       0.74      0.62      0.68        27


    accuracy                           0.84       100
    macro avg      0.81      0.77      0.79       100
 weighted avg      0.83      0.84      0.83       100
```

Read the row for your positive class (Churned). Accuracy appears but is not the primary metric — precision, recall, and F1 for the positive class are.

5.6 Choosing Your Metric — The Business Trade-Off

Precision and recall are inversely linked for a given model. Increasing the decision threshold makes the model more conservative — it predicts positive less often, which raises precision and lowers recall. Decreasing the threshold does the opposite.

The right metric depends on the cost of each type of error:

Scenario	False Positive Cost	False Negative Cost	Prefer
Churn prevention outreach	Low (wasted marketing contact)	High (lost customer)	Recall
Fraud detection (card freeze)	Medium (customer frustration)	High (financial loss)	F1 or Recall
Loan default prediction	High (declining a creditworthy customer)	High (approving a defaulter)	F1
Cancer screening	Low (unnecessary follow-up test)	Very high (missed diagnosis)	Recall

This is a business decision, not a statistical one. Define the costs of each error type with stakeholders before choosing the model's evaluation target.

5.7 Comparing Models

Two models on the same dataset should be compared on the same metric. Fit both, extract the relevant metric from `classification_report`, and prefer the one that scores higher on the metric you have chosen.

```
from sklearn.metrics import f1_score

lr_f1   = f1_score(y_test, y_pred_lr)
tree_f1 = f1_score(y_test, y_pred_tree)

print(f"Logistic Regression F1: {lr_f1:.3f}")
print(f"Decision Tree F1:       {tree_f1:.3f}")
```

A model with higher F1 on the chosen positive class wins — unless there are other considerations (interpretability, inference speed, training cost). In many business settings, a slightly lower-performing model that can be explained to a regulator or executive is preferable to a higher-performing black box.

Chapter 5 Summary

Concept	Key Takeaway
Binary classification	Two-class target — model predicts probability, then classifies at a threshold
Logistic regression	Linear model + sigmoid — outputs probability; fast, interpretable, well-calibrated
`max_iter=1000`	Always set on `LogisticRegression` — default 100 often causes convergence warnings
Decision tree	Splits on feature thresholds — interpretable, handles nonlinearity, overfits without depth limit
`max_depth`	Most important tree hyperparameter — tune via cross-validation (Chapter 6)
Feature importance	Tree-native measure of each feature's contribution to impurity reduction
Accuracy	Misleading on imbalanced data — a model predicting the majority class always achieves it
Confusion matrix	TP, TN, FP, FN — the raw breakdown of every prediction outcome
Precision	Of predicted positives, how many are correct — optimise when false alarms are costly
Recall	Of actual positives, how many were found — optimise when misses are costly
F1 score	Harmonic mean of precision and recall — use when both types of error matter
Classification report	One call for all metrics — read the row for your positive class
Threshold trade-off	Precision and recall trade off at every threshold — the business defines which error is worse

Review Questions

1. A fraud detection model achieves 99.2% accuracy on a dataset where 0.8% of transactions are fraudulent. A colleague declares this an excellent model. What is the flaw in that assessment? What metric would you report instead, and why?

2. Explain the difference between precision and recall in the context of a customer churn model. If the cost of missing a churner far exceeds the cost of a false alarm, which metric should you optimize and why?
3. A decision tree with `max_depth=None` achieves Train accuracy = 1.00 and Test accuracy = 0.61. What is happening, and what is the single most effective fix?
4. Write the code to fit a `LogisticRegression` model, predict on the test set, and print a `classification_report`. Assume `X_train_processed`, `X_test_processed`, `y_train`, and `y_test` are already defined.
5. Your logistic regression model has precision = 0.80 and recall = 0.45 for the positive class. A teammate suggests lowering the decision threshold from 0.5 to 0.3. What effect will this have on precision and recall? Under what business conditions would this change be worth making?

Chapter 6 — Model Tuning and Validation

A model evaluated on a single test set gives one estimate of performance — and that estimate depends heavily on which rows ended up in the test set by chance. This chapter solves that problem with cross-validation, introduces random forests as a more robust alternative to single decision trees, and establishes a systematic process for finding the hyperparameter values that make a model perform best.

6.1 The Problem with a Single Train/Test Split

When you split a dataset once — say, 80% train and 20% test — you get one performance estimate. That estimate has variance. On a lucky split, the test set happens to contain easy-to-predict rows, and the model looks better than it is. On an unlucky split, the test set is harder than average, and the model looks worse. With a small dataset (a few hundred rows), the difference between a lucky and unlucky split can be several percentage points in F1.

The deeper issue: during hyperparameter tuning, if you evaluate multiple configurations on the same test set and pick the best one, you are implicitly fitting to the test set. The test set loses its value as an unbiased estimate of real-world performance. The more decisions you make using test set results, the more optimistic those results become.

Cross-validation solves both problems.

6.2 K-Fold Cross-Validation

K-fold cross-validation divides the training data into `k` equal parts (folds). The model is trained `k` times — each time using `k−1` folds for training and the remaining fold for evaluation. The result is `k` performance estimates, one per fold.

```
Data: [fold 1] [fold 2] [fold 3] [fold 4] [fold 5]

Run 1: train on folds 2–5, evaluate on fold 1
Run 2: train on folds 1, 3–5, evaluate on fold 2
Run 3: train on folds 1–2, 4–5, evaluate on fold 3
Run 4: train on folds 1–3, 5, evaluate on fold 4
Run 5: train on folds 1–4, evaluate on fold 5
```

You then average the `k` scores for a stable estimate of model performance, and compute the standard deviation to understand how much it varies across folds.

`cross_val_score`

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000, random_state=42)

scores = cross_val_score(
    lr, X_train_processed, y_train,
    cv=5,
    scoring='f1'
)

print(f"F1 per fold: {scores.round(3)}")
print(f"Mean F1:     {scores.mean():.3f}")
print(f"Std F1:      {scores.std():.3f}")
```

A low standard deviation (< 0.05) means the model performs consistently across different subsets of the data — a sign of stability. A high standard deviation means performance depends heavily on which data the model sees, which is a concern.

Choosing k

5-fold and 10-fold are the standard choices. 5-fold is faster; 10-fold gives a more stable estimate. For datasets with fewer than 500 rows, 10-fold is worth the extra compute time. For very small datasets (< 100 rows), leave-one-out cross-validation (k = n) is used.

Cross-Validation and the Test Set

Cross-validation runs entirely within the training data. The test set is still held out and not touched until the very end — one final evaluation after all tuning is complete. Cross-validation replaces the need to use the test set during model selection; it does not replace the test set.

6.3 Random Forests

A random forest is an ensemble of decision trees. Each tree is trained on a slightly different version of the data, and predictions are made by majority vote (classification) or averaging (regression) across all trees. The result is more stable and more accurate than any single tree.

Why Ensembles Work: Bagging

Each tree in the forest is trained on a **bootstrap sample** — a random sample of the training data drawn with replacement, the same size as the original dataset. Roughly 63% of rows appear in each bootstrap sample; the remaining 37% are out-of-bag (OOB) and can be used for an internal validation estimate.

Averaging many trees trained on slightly different data reduces variance — the instability that makes single decision trees sensitive to small changes in the training set. Each tree may overfit its bootstrap sample, but the errors average out across trees.

Feature Randomness

At each split in each tree, the random forest considers only a random subset of features — `max_features` features out of the total. This prevents all trees from making the same splits on the same dominant features, ensuring diversity in the ensemble.

Fitting a Random Forest

```
from sklearn.ensemble import RandomForestClassifier


rf = RandomForestClassifier(
    n_estimators=100,    # number of trees
    max_depth=None,      # trees grow fully by default (bagging handles overfitting)
    max_features='sqrt', # sqrt(n_features) considered at each split (default for
classification)
    random_state=42
)
rf.fit(X_train_processed, y_train)
```

Key hyperparameters:

Parameter	What it controls	Typical range
`n_estimators`	Number of trees	100–500
`max_depth`	Maximum tree depth	None (default), 5–20
`max_features`	Features considered per split	`'sqrt'` (default), `'log2'`, a fraction
`min_samples_leaf`	Minimum samples per leaf	1 (default), 2–10

Feature Importance

Random forest feature importances average impurity reduction across all trees — more stable than a single tree's importances.

```
importances = pd.DataFrame({
    'feature':    preprocessor.get_feature_names_out(),
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)
```

6.4 Hyperparameter Tuning

Model parameters (coefficients, tree split thresholds) are learned from data during `.fit()`. **Hyperparameters** (number of trees, maximum depth, regularization strength) are set before training and control the learning process. Cross-validation is used to find the best hyperparameter values.

GridSearchCV

`GridSearchCV` evaluates every combination of hyperparameter values in a specified grid, using cross-validation for each.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth':    [3, 5, 10, None],
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1     # use all available CPU cores
)
grid_search.fit(X_train_processed, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV F1:      {grid_search.best_score_:.3f}")
```

The grid above evaluates 3 × 4 = 12 combinations, each with 5-fold CV — 60 model fits in total. `n_jobs=-1` parallelizes this across cores. Larger grids take proportionally more time.

RandomizedSearchCV

When the hyperparameter space is large, exhaustive grid search is impractical. `RandomizedSearchCV` samples `n_iter` random combinations instead of evaluating every one.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_dist = {
    'n_estimators':     randint(50, 500),
    'max_depth':        [3, 5, 10, 15, None],
    'min_samples_leaf': randint(1, 10),
}

rand_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_dist,
    n_iter=20,
    cv=5,
    scoring='f1',
    random_state=42,
    n_jobs=-1
)
rand_search.fit(X_train_processed, y_train)
```

`RandomizedSearchCV` with `n_iter=20` evaluates 20 random combinations — a small fraction of the full space — and typically finds near-optimal values in a fraction of the time. Use it when the grid has more than three or four parameters, or when the value ranges are continuous.

6.5 The Bias-Variance Trade-Off

Every model makes two types of errors:

Bias — systematic error from the model being too simple to capture the true relationship. A linear model fit to strongly nonlinear data has high bias. It underfits.

Variance — error from the model being too sensitive to the specific training data. A deep, unconstrained decision tree has high variance. It overfits.

The trade-off: reducing bias (adding complexity) typically increases variance, and vice versa. The goal is to find the point where total error is minimized.

Model	Bias	Variance
Linear / logistic regression	Higher	Lower
Single deep decision tree	Lower	Higher
Random forest (many shallow trees)	Lower	Much lower
Random forest (many deep trees)	Lowest	Low

Random forests sit in a favorable position: they have low bias (each tree can learn complex patterns) and low variance (averaging across trees reduces instability). This is why they outperform single trees on almost every real dataset.

6.6 The Final Model Protocol

Hyperparameter tuning creates a risk: if you evaluate many configurations on the same test set and choose the best, the test set has leaked into your model selection process. The correct protocol is strict:

1. **Split** the full dataset into train and test sets — the test set is locked away.
2. **Tune** using `GridSearchCV` or `RandomizedSearchCV` with cross-validation on the training set only.
3. **Refit** the best model on the full training set (not just one CV fold).
4. **Evaluate** on the test set — once, at the very end.

```
# Step 3: refit is automatic when refit=True (the default in GridSearchCV)
best_model = grid_search.best_estimator_
# best_estimator_ is already refit on the full training set

# Step 4: evaluate on the test set — once
y_pred_final = best_model.predict(X_test_processed)
print(classification_report(y_test, y_pred_final, target_names=['Retained', 'Churned']))
```

If your test score is substantially lower than the best CV score from grid search, the hyperparameter search overfit to the training data — the grid was too fine, or the dataset is too small for the number of hyperparameters explored.

Chapter 6 Summary

Concept	Key Takeaway
Single split variance	One test set gives one estimate — lucky or unlucky; cross-validation stabilizes it
K-fold CV	Trains k models, averages k scores — low std means consistent performance
`cross_val_score`	Runs CV in one call — use within training data only, never touch test set
Bagging	Each tree trains on a bootstrap sample — averaging reduces variance
Feature randomness	Random subset of features per split — ensures tree diversity in the ensemble
`n_estimators`	More trees = more stable predictions; diminishing returns above 200–300
`max_depth` in forests	Less critical than in single trees — bagging handles overfitting
Hyperparameters vs. parameters	Hyperparameters are set before training; parameters are learned during `.fit()`
`GridSearchCV`	Exhaustive search — evaluates every combination in the grid with CV
`RandomizedSearchCV`	Samples n_iter random combinations — faster for large parameter spaces
Bias-variance trade-off	Simple models underfit (high bias); complex models overfit (high variance)
Final model protocol	Tune on training data → refit on full training set → evaluate test set once

Review Questions

1. A colleague evaluates five different model configurations on the same test set and picks the one with the highest F1. Why is this test F1 no longer a reliable estimate of real-world performance? What should they have done instead?

2. You run 5-fold cross-validation on a random forest and get F1 scores of `[0.71, 0.58, 0.74, 0.72, 0.69]`. What does the score of 0.58 in fold 2 suggest? Is this model reliable enough to deploy? What information would you want before deciding?
3. Explain why a random forest with 100 trees is almost always more accurate than a single decision tree, even when both are allowed to grow to the same depth.
4. Write the `GridSearchCV` code to tune `n_estimators` (100, 200) and `max_depth` (5, 10, None) for a `RandomForestClassifier`. Use 5-fold CV with F1 scoring. Print the best parameters and best CV score.
5. After running `GridSearchCV`, your best CV F1 is 0.74 but your test set F1 is 0.61. What likely went wrong, and what would you do differently?

Chapter 7 — End-to-End Pipelines and Visualization

By Week 6 the workflow involves several independent objects: a `ColumnTransformer`, a model, a grid search, separate `.fit_transform()` and `.transform()` calls. Each step depends on the previous one, and a mistake at any step — fitting the scaler on test data, forgetting to transform before predicting — produces wrong results without raising an error. The `Pipeline` object eliminates that class of mistake entirely. This chapter wraps the full workflow into a single object, tunes it as a unit, and introduces Plotly Express for communicating results to stakeholders.

7.1 The Case for Pipelines

Consider the manual workflow:

```
# Manual — four places to make a mistake
preprocessor.fit(X_train)
X_train_proc = preprocessor.transform(X_train)
X_test_proc  = preprocessor.transform(X_test)   # easy to accidentally fit here instead
model.fit(X_train_proc, y_train)
y_pred = model.predict(X_test_proc)
```

Each step is a separate call. A student who writes `preprocessor.fit_transform(X_test)` instead of `preprocessor.transform(X_test)` has introduced data leakage — silently. The code runs, the predictions look plausible, and the error is invisible.

A `Pipeline` enforces the correct sequence automatically:

```
# Pipeline — one object, no sequencing mistakes possible
pipe = Pipeline(steps=[('pre', preprocessor), ('model', model)])
pipe.fit(X_train, y_train)        # fits preprocessor on X_train, transforms, fits model
y_pred = pipe.predict(X_test)     # transforms X_test using training params, predicts
```

`.fit()` on a pipeline fits each step in order and passes the output forward. `.predict()` applies `.transform()` to every step except the last, then calls `.predict()` on the final estimator. The split-before-fit rule is still your responsibility — the pipeline does not perform the train/test split — but every preprocessing and model step executes in the correct sequence automatically.

Additional Benefits

`GridSearchCV` on the pipeline: you tune the full workflow, not just the model. If you tune only the model, the preprocessor's parameters (e.g., the scaler's choice) are not searched. Tuning the pipeline means every parameter — preprocessing and model — is considered together.

Deployment: a fitted pipeline is a single serializable object. Save it, load it, and call `.predict()` on raw new data — the pipeline handles transformation automatically. There is no separate preprocessor to keep in sync.

7.2 Building a sklearn Pipeline

A `Pipeline` takes a list of `(name, estimator)` tuples. Every step except the last must implement `.transform()`. The last step must implement `.predict()` or `.fit()`.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier

# Define the preprocessor (same ColumnTransformer as before)
preprocessor = ColumnTransformer(transformers=[
    ('ord',  OrdinalEncoder(categories=[['Month-to-Month', 'One Year', 'Two Year']]),
             ['contract_type']),
    ('cat',  OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=False),
             ['customer_segment']),
    ('num',  StandardScaler(),
             ['tenure_months', 'monthly_charges', 'num_products']),
    ('pass', 'passthrough',
             ['tech_support']),
])

# Build the pipeline
pipe = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model',        RandomForestClassifier(n_estimators=100, random_state=42)),
])

# Fit and predict — no separate transform calls needed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

The step names (`'preprocessor'` , `'model'`) are arbitrary strings — choose names that are readable. They become the prefix for hyperparameter tuning.

Accessing Pipeline Components

```
# Access the fitted preprocessor
fitted_preprocessor = pipe.named_steps['preprocessor']
feature_names = fitted_preprocessor.get_feature_names_out()

# Access the fitted model
fitted_model = pipe.named_steps['model']
importances  = fitted_model.feature_importances_
```

7.3 Pipeline + GridSearchCV

When you pass a pipeline to `GridSearchCV`, parameter names follow the pattern `stepname__parametername` (two underscores). This is the only change from a regular grid search.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth':    [5, 10, None],
}

grid_search = GridSearchCV(
    pipe,
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)   # raw X_train — pipeline handles transformation

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV F1:      {grid_search.best_score_:.3f}")

# Predict with the best pipeline — raw X_test, no separate transform needed
y_pred_final = grid_search.best_estimator_.predict(X_test)
```

Preprocessing parameters can also be tuned through the pipeline:

```
param_grid = {
    'preprocessor__num__with_mean': [True, False],  # StandardScaler parameter
    'model__n_estimators':          [100, 200],
}
```

The naming depth follows the nesting: `preprocessor` (pipeline step) → `num` (ColumnTransformer transformer name) → `with_mean` (StandardScaler parameter).

7.4 Predicted Probabilities as Business Output

`.predict()` returns class labels (0 or 1). For business use, predicted **probabilities** are more useful — they let you rank customers by risk level and prioritize intervention.

```
# Get churn probability for each customer
churn_proba = pipe.predict_proba(X_test)[:, 1]  # column 1 = P(churned)

# Segment into risk tiers
risk_bins   = [0, 0.25, 0.50, 0.75, 1.01]
risk_labels = ['Low', 'Medium', 'High', 'Critical']

results = X_test.copy().reset_index(drop=True)
results['churn_probability'] = churn_proba
results['risk_tier']         = pd.cut(churn_proba, bins=risk_bins, labels=risk_labels)
results['actual_churned']    = y_test.values

print(results[['tenure_months', 'contract_type', 'churn_probability', 'risk_tier']].head(10))
```

A ranked list of at-risk customers is more actionable than a single accuracy number. The retention team works the Critical tier first, then High — and the model tells them which customers to call, not just what percentage will churn.

7.5 Visualization with Plotly Express

Plotly Express creates interactive charts — hover to see values, click to filter, zoom in — that are far more effective in presentations than static matplotlib figures. Every chart is a single function call.

Setup

```
import plotly.express as px
```

Bar Chart

```
# Churn rate by contract type
churn_by_contract = (df.groupby('contract_type')['churned']
                        .mean()
                        .reset_index()
                        .rename(columns={'churned': 'churn_rate'}))

fig = px.bar(
    churn_by_contract,
    x='contract_type',
    y='churn_rate',
    title='Churn Rate by Contract Type',
    labels={'churn_rate': 'Churn Rate', 'contract_type': 'Contract Type'},
    color='churn_rate',
    color_continuous_scale='Reds',
)
fig.update_layout(yaxis_tickformat='.0%')
fig.show()
```

Histogram

```
# Distribution of predicted churn probabilities
fig = px.histogram(
    results,
    x='churn_probability',
    color='risk_tier',
    nbins=30,
    title='Distribution of Predicted Churn Probability',
    labels={'churn_probability': 'Predicted Churn Probability'},
    category_orders={'risk_tier': ['Low', 'Medium', 'High', 'Critical']},
)
fig.show()
```

Box Plot

```
# Churn probability by customer segment
fig = px.box(
    results,
    x='customer_segment',
    y='churn_probability',
    color='customer_segment',
    title='Churn Probability by Customer Segment',
    labels={'churn_probability': 'Predicted Churn Probability'},
)
fig.show()
```

Scatter Plot

```
# Tenure vs. churn probability, colored by contract type
fig = px.scatter(
    results,
    x='tenure_months',
    y='churn_probability',
    color='contract_type',
    hover_data=['monthly_charges', 'risk_tier'],
    title='Tenure vs. Churn Probability',
    labels={
        'tenure_months':    'Tenure (months)',
        'churn_probability': 'Predicted Churn Probability',
        'contract_type':     'Contract Type',
    },
)
fig.add_hline(y=0.5, line_dash='dash', line_color='red',
              annotation_text='Default threshold (0.5)')
fig.show()
```

7.6 From Model Score to Business Recommendation

A model is not the deliverable. The deliverable is a decision — who to contact, what to offer, how to allocate the retention budget. The model output is the evidence that justifies the decision.

A complete business presentation of a churn model includes:

1. The problem statement. "We lose approximately X% of customers each quarter. Each churned customer represents $Y in lost annual revenue. We built a model to identify which customers are most likely to leave before they do."

2. The model's performance. "The model correctly identifies Z% of customers who will churn (recall), with W% of its alerts being true positives (precision). On our test set of 80 customers, it flagged N customers for retention outreach — of whom M actually churned."

3. The actionable output. Present the risk tier breakdown. Show how many customers fall into each tier. Recommend a specific action for each tier.

4. The top drivers. "The three strongest predictors of churn are: contract type (month-to-month customers churn at 3× the rate of annual contract customers), customer tenure (first-year customers are highest risk), and monthly charges (higher-spend customers churn more)."

5. The recommendation. "Prioritise retention outreach for the N customers in the Critical tier. Consider offering contract upgrades to month-to-month customers in months 1–12 of tenure."

This structure works for any supervised model, not just churn. Replace the domain-specific details and the structure transfers.

Chapter 7 Summary

Concept	Key Takeaway
`Pipeline`	Chains preprocessing + model — `.fit()` and `.predict()` handle all steps in order
Leakage prevention	Pipeline calls `.transform()` (not `.fit_transform()`) on test data automatically
`named_steps`	Access any fitted pipeline component by its step name
`step__param` naming	Two underscores — required for `GridSearchCV` on a pipeline
Raw data in, predictions out	Pipeline accepts raw `X_train` / `X_test` directly — no separate transform calls
`predict_proba`	Returns class probabilities — more useful for ranking and segmentation than binary labels
Risk tiers	`pd.cut` converts continuous probabilities into business-interpretable categories
`px.bar`	Bar chart in one call — use for categorical comparisons
`px.histogram`	Distribution view — use to show how predicted probabilities are spread
`px.box`	Distribution by group — use to compare model output across segments
`px.scatter`	Two numeric variables + optional color — use for relationship exploration
Business presentation	Problem → performance → actionable output → drivers → recommendation

Review Questions

1. A colleague builds a `ColumnTransformer`, fits it separately on training data, transforms both train and test sets manually, then fits a model. They argue a pipeline is unnecessary because they are already following the correct sequence. Give two concrete reasons why the pipeline is still the better choice.

2. You have a pipeline with steps named `'prep'` and `'classifier'`. The classifier is a `RandomForestClassifier`. Write the `param_grid` dictionary to search over `n_estimators` (100, 300) and `max_depth` (5, None) using `GridSearchCV`.
3. Your pipeline's `.predict()` returns `[0, 1, 0, 0, 1, ...]`. Your colleague wants to know which of the 200 customers in the test set is most at risk of churning. Explain what additional method to call and what its output looks like.
4. Explain why `pipe.fit(X_train, y_train)` does not cause data leakage even though the preprocessor sees `X_train` and the model also trains on `X_train`.
5. A `px.scatter` chart shows that high-tenure customers with month-to-month contracts still have moderate predicted churn probabilities. Your model assigns them churn_probability ≈ 0.40. What business question does this raise, and how would you investigate it using the model's output?

Chapter 8 — Unsupervised Learning: Clustering

Every model built in Weeks 4–7 required a target variable — a column of known outcomes used to train and evaluate the model. Clustering removes that requirement. There is no label, no F1 score, no ground truth to measure against. Instead of predicting, you are discovering: finding natural groups in data that share similar characteristics. This chapter introduces KMeans and hierarchical clustering, the tools for choosing the right number of groups, and the judgment required to turn mathematical clusters into business segments.

8.1 Unsupervised Learning

Unsupervised learning finds structure in data without predefined labels. The two most common forms in business are:

- **Clustering** — grouping observations so that members of the same group are more similar to each other than to members of other groups. Used for customer segmentation, document grouping, anomaly detection.
- **Dimensionality reduction** — compressing many features into fewer dimensions while preserving structure. Used for visualization and as a preprocessing step before supervised modelling.

This chapter covers clustering. The core business use case is **market segmentation**: given a dataset of customer behaviour with no predefined categories, find the natural groupings and describe what makes each group distinct.

What Changes Without a Target

In supervised learning, the model's quality is measured against known labels — precision, recall, R^2. In unsupervised learning there are no labels to measure against. Quality is assessed by:

- **Within-cluster cohesion** — members of the same cluster should be similar to each other
- **Between-cluster separation** — different clusters should be distinct from each other
- **Business interpretability** — the clusters should correspond to groups a stakeholder can act on

The last criterion is the most important and the hardest to quantify. A mathematically optimal clustering that produces groups without interpretable business meaning is useless.

8.2 KMeans Clustering

KMeans partitions `n` observations into `k` clusters. Each observation belongs to the cluster whose centroid (mean) is nearest.

The Algorithm

1. **Initialize** — place `k` centroids randomly in the feature space
2. **Assign** — assign each observation to the nearest centroid (by Euclidean distance)
3. **Update** — recalculate each centroid as the mean of its assigned observations
4. **Repeat** steps 2–3 until assignments stop changing (convergence)

```
from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X_scaled)

labels     = km.labels_       # cluster assignment for each row
centroids  = km.cluster_centers_  # centroid coordinates
inertia    = km.inertia_      # sum of squared distances to nearest centroid
```

`n_init=10` runs the algorithm 10 times with different random initializations and keeps the best result. KMeans is sensitive to initialization — running it once may land in a poor local minimum.

Distance and Scale

KMeans assigns observations to the nearest centroid using Euclidean distance. If one feature ranges from 0 to 10,000 (annual spend) and another from 1 to 5 (number of products), the spend feature will dominate the distance calculation. The number of products becomes invisible to the model.

Always scale features before running KMeans. `StandardScaler` is the standard choice. This is not optional — unscaled KMeans produces clusters driven entirely by whichever features have the largest numeric range.

8.3 Choosing the Number of Clusters

KMeans requires you to specify `k` in advance. Two methods help identify the right value.

The Elbow Method

Run KMeans for a range of `k` values and plot the **inertia** (within-cluster sum of squared distances) for each. As `k` increases, inertia decreases — but the rate of decrease slows. The "elbow" — the point where adding another cluster yields diminishing returns — is a reasonable choice for `k`.

```
inertias = []
k_range  = range(2, 11)

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

import matplotlib.pyplot as plt
plt.plot(k_range, inertias, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

The elbow is often not a sharp point — it is a region where the curve bends. When the elbow is ambiguous, use the silhouette score to decide.

The Silhouette Score

The silhouette score measures how well each observation fits its assigned cluster relative to the nearest alternative cluster. It ranges from −1 to 1:

- **+1** — the observation is well within its cluster and far from others (ideal)
- **0** — the observation is on the boundary between two clusters
- **−1** — the observation would fit better in a neighbouring cluster (misclassified)

```
from sklearn.metrics import silhouette_score

silhouettes = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    silhouettes.append(silhouette_score(X_scaled, labels))

plt.plot(k_range, silhouettes, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Score by k')
plt.show()
```

Choose `k` where the silhouette score is highest — this indicates the best-defined clusters. If the silhouette and elbow methods disagree, let business interpretability break the tie.

8.4 Interpreting and Profiling Clusters

A cluster label (0, 1, 2) is not a business segment. Profiling converts labels into descriptions.

```
# Add cluster labels to the original (unscaled) DataFrame
df['cluster'] = km.labels_

# Compute the mean of each feature within each cluster
profile = df.groupby('cluster').mean().round(2)
print(profile)
```

Read across each row: cluster 0 customers have low recency (purchased recently), high frequency, and high average order value — these are loyalists. Cluster 1 has high recency, low frequency, low order value — these are at-risk or lapsed customers.

Naming clusters. Assign a business-meaningful name to each cluster based on its profile. Names like "High-Value Loyalists", "Occasional Buyers", and "Lapsed Customers" communicate the segmentation to stakeholders without requiring them to read a statistics table.

Cluster size. Check that clusters are reasonably sized. A cluster containing 2% of customers may be a genuine niche or an artefact of noise. If one cluster contains 90% of customers, `k` may be too low — the model has not found meaningful sub-groups.

8.5 Hierarchical Clustering

Hierarchical clustering builds a tree of nested clusters without requiring you to specify `k` in advance. The most common approach is **agglomerative** (bottom-up):

1. Start with each observation as its own cluster (n clusters)
2. Merge the two closest clusters
3. Repeat until all observations are in one cluster

The result is a **dendrogram** — a tree diagram showing which clusters merge at what distance.

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Compute linkage for the dendrogram
Z = linkage(X_scaled, method='ward')

plt.figure(figsize=(10, 5))
dendrogram(Z, truncate_mode='lastp', p=20, show_leaf_counts=True)
plt.xlabel('Sample index or (cluster size)')
plt.ylabel('Distance')
plt.title('Hierarchical Clustering Dendrogram')
plt.show()
```

Reading the Dendrogram

The height at which two branches merge reflects the distance between those clusters. Cutting the dendrogram at a chosen height produces a specific number of clusters — the number of vertical lines that the horizontal cut intersects.

The largest vertical gaps in the dendrogram suggest natural break points — clusters that are well-separated. Cut at the largest gap that gives a business-useful number of groups.

Fitting AgglomerativeClustering

```
agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
agg_labels = agg.fit_predict(X_scaled)
```

`linkage='ward'` minimizes the total within-cluster variance at each merge step — it produces compact, roughly equal-sized clusters and is the standard starting choice.

8.6 Choosing Between KMeans and Hierarchical

Criterion	KMeans	Hierarchical
k required in advance?	Yes	No (cut dendrogram after)
Scales to large datasets	Yes (fast)	No ($O(n^2)$ memory)
Cluster shape assumed	Spherical (Euclidean distance)	Flexible
Reproducibility	Varies with `random_state`	Deterministic
Interpretability	Centroid table	Dendrogram shows merge structure
Best for	Large datasets, known approximate k	Small datasets, exploring natural hierarchy

For datasets up to a few thousand rows, fit both and compare. For datasets above 10,000 rows, KMeans is the practical choice — hierarchical clustering becomes too slow.

8.7 Cluster Stability and Limitations

Stability

Run KMeans several times with different random seeds and check whether the cluster profiles remain consistent. Stable profiles (same characteristics, possibly different integer labels) indicate the clusters reflect real structure. Unstable profiles suggest the data does not have clearly defined natural groups at the chosen `k`.

Limitations

No ground truth. There is no equivalent of F1 to tell you the clusters are correct. Silhouette score and inertia measure mathematical properties, not business value. Always validate clusters with domain knowledge.

Correlation with features not used. After clustering, check whether the clusters differ on variables you did not include in the clustering. If two clusters have nearly identical profiles on all external variables, they may not represent meaningfully different customer groups.

k selection is subjective. The elbow method and silhouette score give guidance, but the final choice of `k` involves judgment. Three versus four segments may both be mathematically reasonable — the right choice is the one that produces groups the business can act on differently.

Chapter 8 Summary

Concept	Key Takeaway
Unsupervised learning	No target variable — discovering structure, not predicting outcomes
KMeans	Assigns observations to k nearest centroids — fast, requires k in advance
`n_init=10`	Run 10 random starts, keep best — avoids poor local minima
Scale before clustering	KMeans uses Euclidean distance — unscaled features dominate by range, not importance
Inertia (elbow method)	Decreases with k — look for the bend where gains diminish
Silhouette score	Range −1 to 1 — higher is better; choose k where score peaks
Cluster profiling	`groupby('cluster').mean()` — describes each cluster in original feature units
Cluster naming	Translate numeric profiles into business-meaningful labels
Hierarchical clustering	Bottom-up merging — dendrogram shows structure; no k required upfront
Dendrogram cut	Cut at the largest gap — the number of intersections is the number of clusters
`linkage='ward'`	Minimizes within-cluster variance — standard starting choice
No ground truth	Silhouette measures cohesion, not correctness — validate with domain knowledge

Review Questions

1. A colleague applies KMeans to a customer dataset with features `annual_spend` (range $500–$50,000) and `number_of_purchases` (range 1–24) without scaling. The resulting clusters are almost entirely determined by `annual_spend`. Why does this happen, and how do you fix it?

2. You run the elbow method on a dataset and get inertias of 4200, 2800, 1900, 1500, 1300, 1200 for k = 2 through 7. Where is the elbow, and what k would you choose? What would you check next?

3. Explain the difference between inertia and silhouette score as measures of clustering quality. Under what circumstances might they suggest different values of k?

4. You fit a KMeans model with k = 4 and find that cluster 2 contains 87% of all observations. What does this tell you about the clustering, and what would you try next?
5. Your hierarchical clustering dendrogram shows two large branches that merge at a height of 12, with all other merges occurring below height 4. What does this pattern suggest about the natural structure of the data?

Chapter 9 — Final Project

9.1 What the Final Project Is About

Every chapter in this course has given you a piece of the data science puzzle: how to load and explore data, how to clean and encode it, how to train and evaluate models, how to tune them properly, how to organize your work into pipelines, how to communicate findings to different audiences. The final project puts all of those pieces together in one place.

You will choose a real dataset, define a meaningful analytical question, work through the complete data science workflow from raw data to insight, and present your findings in both a technical notebook and a plain-language executive summary. The goal is not perfection — it is demonstration of sound judgment at each stage of the process.

Unlike the weekly labs, there is no single correct answer here. Two students can use the same dataset, ask different questions, and both produce excellent work. What matters is that your choices are deliberate and explained.

> 💼 **Business Context:** Real data science projects almost never have a tidy answer key. The skill employers value most is the ability to define the right question, make defensible decisions under uncertainty, and explain what the results mean to someone who was not in the room when you built the model.

9.2 Dataset Selection

Choosing a good dataset is itself a data science skill. A dataset that is too clean teaches you nothing about preprocessing. A dataset that is too large or too messy may consume all your time before you reach any analysis. A dataset with no interesting signal will produce a model that tells you nothing.

Criteria for a good final project dataset:

Your dataset should have at least 200 rows (more is fine) and at least 5 features, including a mix of numeric and categorical variables. If you are doing supervised learning, you need a clear target variable — something meaningful to predict. If you are doing unsupervised learning (clustering), you need enough features to make segmentation interesting.

The dataset should come from a domain you understand or can research. You will need to interpret your results in context, which requires knowing what the numbers mean. A dataset about hospital readmission rates is interesting, but only if you can explain why a particular feature might matter clinically.

Avoid datasets that are heavily pre-cleaned for teaching purposes, such as the famous Titanic or Iris datasets. Reviewers and employers recognize these immediately. Choose something where you had to make real preprocessing decisions.

Suggested sources:

Kaggle (kaggle.com/datasets) offers thousands of real-world datasets across industries. The UCI Machine Learning Repository (archive.ics.uci.edu) has well-documented datasets with known characteristics. Open government portals (Statistics Canada, data.gov, data.gov.uk) offer datasets from healthcare, education, transportation, and finance. Your instructor may also suggest domain-specific sources relevant to your program.

Dataset confirmation: Before committing, answer these questions for yourself — if any answer is "I don't know," keep looking:

- What does each row represent? (A customer, a transaction, a patient visit, a housing unit?)
- What question am I trying to answer with this data?
- Is there a target variable, or am I exploring structure?
- Do I understand what the features mean in plain language?
- Are there enough rows to train, validate, and test a model?

9.3 The Project Workflow

The final project follows the same workflow you have practised in the weekly labs, but now you own every decision. The sections below describe what is expected at each stage.

9.3.1 Problem Definition

Before writing any code, write one paragraph explaining what you are trying to accomplish and why it matters. This is not a throwaway step — it is what keeps your analysis focused when you are three hours deep in feature engineering and have forgotten what the original question was.

Your problem statement should name the domain, identify the question (predictive or exploratory), and specify what a useful result would look like. "I want to predict whether a loan applicant will default, so that a lender can flag high-risk applications for manual review before approval" is a problem statement. "I am going to build a model on this dataset" is not.

9.3.2 Exploratory Data Analysis

EDA is not optional decoration. It is where you learn whether your dataset can actually answer your question, discover data quality issues before they corrupt your model, and identify which features are likely to matter.

Your EDA section should cover the shape and types of your data, missing value counts and your handling strategy, distributions of key numeric features (with Plotly visualizations), counts and proportions of categorical features, and the relationship between features and the target (if supervised). By the end of EDA, you should be able to describe your dataset to someone who has never seen it, and justify why it can address your problem statement.

9.3.3 Preprocessing

Every dataset requires preprocessing decisions. Document yours. For each feature, state what you did and why — scaled, encoded, imputed, or dropped — and how you handled it (which transformer). Your ColumnTransformer should be defined and fit only on training data. If you are doing clustering, your scaler should be fit on the full dataset (since there is no split).

Common preprocessing errors to avoid: fitting scalers or encoders on the full dataset before splitting (data leakage), dropping rows with missing values without checking whether missingness is random, and one-hot encoding ordinal variables without thinking about whether order matters.

9.3.4 Modelling

For supervised projects, you must train at least two models and compare them properly. Use cross-validation (not a single train/test split) for model selection. If you tune hyperparameters, use GridSearchCV or RandomizedSearchCV inside the cross-validation — never tune on the test set. Evaluate your final model on the held-out test set exactly once, at the very end.

For unsupervised projects, you must justify your choice of k (or number of clusters) using at least two methods — for example, the elbow method and silhouette scores. Fit your final model and produce a meaningful interpretation of what each cluster represents in plain language.

Wrap everything in a Pipeline. This is not bureaucracy — it prevents preprocessing leakage, makes your code reproducible, and makes deployment straightforward.

9.3.5 Evaluation and Interpretation

Numbers without interpretation are not findings. For every metric you report, write one sentence explaining what it means for your specific problem. An F1 score of 0.72 on churn prediction means something different from an F1 score of 0.72 on fraud detection — because the costs of false positives and false negatives differ enormously.

For regression, report MAE, RMSE, and R^2. Discuss whether the error is acceptable in context — a $500 MAE on house prices might be excellent; the same error on a grocery receipt prediction would be absurd. For classification, report precision, recall, and F1, and include the confusion matrix. For clustering, report silhouette score and provide named, interpreted segment profiles.

> 🤖 **ML Connection:** The final judgment on a model is never "is the accuracy high?" It is always "is this model useful for the decision it was built to support?" A model with 60% accuracy that correctly identifies the 10% of cases that matter most may be far more valuable than a model with 90% accuracy that misses all of them.

9.3.6 Communication

Your project has two deliverables: a technical notebook and an executive summary. The technical notebook is for a data-literate audience — it should be clean, commented, and reproducible. The executive summary is for a non-technical decision-maker — it should be written in plain language with no code, no jargon, and a clear recommendation.

The executive summary is typically one to two pages. It should state the problem, summarize key findings with supporting visuals (you may paste charts from your notebook), and make a concrete, actionable recommendation. Hedging is fine — "we recommend prioritizing customers in the High Risk tier for retention outreach, though we note this model should be retrained quarterly as customer behaviour evolves" is better than false precision.

9.4 Deliverables

Submit the following three items as a single compressed folder or shared link:

1. Project Notebook (`project_notebook.ipynb`)
A clean, fully executed Jupyter notebook. Every cell should have run successfully from top to bottom with no errors. Markdown cells should explain what each section is doing and why. Code should be readable — use meaningful variable names, avoid redundant cells, and remove dead ends and failed experiments from the final submission (those belong in a scratch notebook, not the deliverable).

2. Executive Summary (`executive_summary.pdf` or `.docx`)
A one-to-two page plain-language report suitable for a non-technical manager. Should include: the business question, key findings (supported by 2–3 charts), your recommendation, and one paragraph on limitations or caveats. No code. No jargon without explanation.

3. Dataset (`data/` folder or source citation)
Include your raw dataset if it is under 50MB and not subject to redistribution restrictions. If the dataset is too large or proprietary, include a `data_source.txt` file with the exact URL and access date.

9.5 Evaluation Criteria

Your project will be evaluated across five dimensions. The rubric below shows how marks are allocated; the full grading guide is available from your instructor.

Dimension	Weight	What We Are Looking For
Problem Definition & EDA	25%	Clear question, thorough exploration, visualizations that reveal insight
Preprocessing & Pipeline	20%	Correct, documented choices; no data leakage; Pipeline used throughout
Modelling & Tuning	25%	At least two models compared; CV used properly; final eval on test set only
Evaluation & Interpretation	15%	Metrics explained in context; results tied back to the business question
Communication	15%	Technical notebook is clean and reproducible; executive summary is clear and actionable

A strong project does not need to achieve state-of-the-art model performance. It needs to demonstrate that you understand why you made each decision and what your results actually mean.

9.6 Common Pitfalls

These are the mistakes that appear most frequently in final projects, and the ones that cost the most marks.

Fitting on the full dataset before splitting. If your scaler or encoder sees the test set during training, your evaluation metrics are optimistic and meaningless. Always split first, then fit transformers on training data only. The Pipeline class exists precisely to enforce this automatically.

Evaluating on the training set. Reporting that your model achieved 95% accuracy without specifying which set is a red flag. Always evaluate on held-out data. If you tuned hyperparameters, that means your test set was never touched until the very final evaluation step.

Choosing the wrong metric for the problem. Accuracy is almost always the wrong primary metric for imbalanced classification problems. If 95% of your observations are one class, a model that predicts that class every time gets 95% accuracy and is completely useless. Use precision, recall, and F1 — and think about which type of error is more costly in your domain.

Describing features without interpreting results. Listing feature importances is not the same as explaining what they mean. "Tenure was the most important feature (importance = 0.34)" should be followed by "this suggests that customers who have been with the company longer are significantly less likely to churn, consistent with the loyalty effect documented in CRM research."

Skipping the executive summary. The executive summary is not optional and it is not a reformatted notebook. It requires you to translate technical findings into plain language — which is often harder than the modelling itself. A one-page document that a manager could actually act on is a more valuable professional skill than a technically perfect model that no one can use.

Over-engineering the model. Students sometimes spend so much time chasing a slightly higher F1 score that they run out of time for EDA, interpretation, and communication. A well-interpreted simple model is better than a poorly understood complex one.

9.7 Project Planning Checklist

Use this checklist to manage your time. Most students underestimate how long EDA and interpretation take, and overestimate how long modelling takes.

✅ Week 1 of Project Period

- [ ] Dataset identified and downloaded
- [ ] Problem statement written (1 paragraph, in plain English)
- [ ] Dataset confirmed: at least 200 rows, 5+ features, clear target or clustering rationale
- [ ] Initial EDA begun: shape, dtypes, missing values, basic distributions

✅ Week 2 of Project Period

- [ ] EDA complete with visualizations (Plotly)
- [ ] Preprocessing decisions documented and implemented in ColumnTransformer
- [ ] First model trained and evaluated with cross-validation
- [ ] Second model trained and compared

✅ Week 3 of Project Period

- [ ] Hyperparameter tuning complete (GridSearchCV or RandomizedSearchCV)
- [ ] Final model evaluated on test set (exactly once)

- [ ] Results interpreted in plain language
- [ ] Model card or model summary written

✅ Final Week

- [ ] Notebook cleaned up — all cells run top to bottom without errors
- [ ] Executive summary drafted (plain language, no jargon, concrete recommendation)
- [ ] All files organized and submission package assembled
- [ ] Peer review or instructor check-in completed (if offered)
- [ ] Submission submitted on time

Chapter 9 Summary

Topic	Key Point
Project purpose	Integrate all course skills in one end-to-end analysis
Dataset selection	Real data, 200+ rows, 5+ features, domain you understand
Problem definition	Write the question before writing code
Workflow	EDA → Preprocessing → Modelling → Evaluation → Communication
Deliverables	Notebook + Executive Summary + Dataset/citation
Evaluation	Problem definition (25%), Preprocessing (20%), Modelling (25%), Interpretation (15%), Communication (15%)
Common pitfalls	Data leakage, wrong metric, training-set evaluation, skipping exec summary
Time management	EDA and communication take longer than expected; budget accordingly

---

License

Creative Commons Attribution 4.0 International (CC BY 4.0)

© 2026 Patrick Dolinger

This work is licensed under the Creative Commons Attribution 4.0 International License.

You are free to:
- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose

Under the following terms:
- **Attribution** — You must give appropriate credit to the author, provide a link to the license, and indicate if changes were made.